

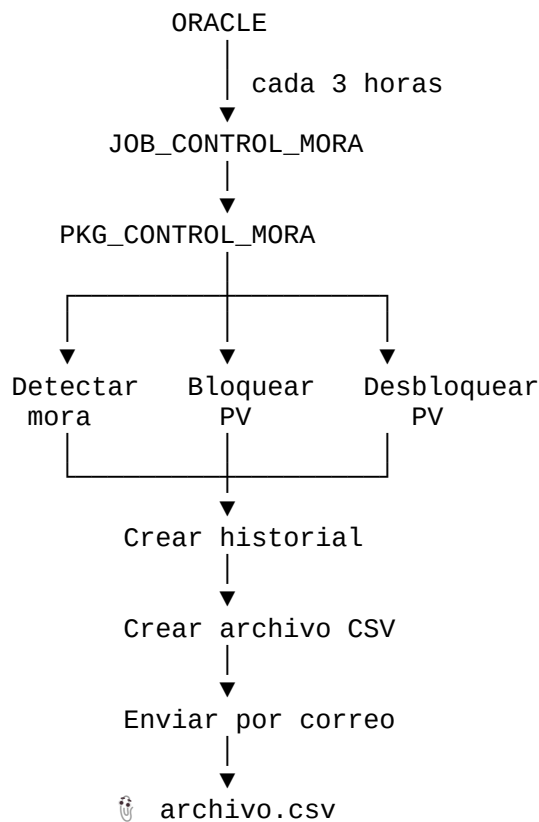
Sí. Vamos a hacerlo **desde cero**, pensando en que es tu primera vez trabajando con UTL_FILE, UTL_SMTP y DBMS_SCHEDULER.

Voy a darte una solución completa y ordenada para que puedas probar cada parte antes de pasar a la siguiente.

Importante: como todavía no tengo el DESC real de tus tablas, usaré los nombres de tablas y campos que hemos venido manejando como ejemplo. Te marcaré claramente qué debes cambiar cuando lo adaptemos a tu base real. No quiero que copies código a producción sin comprobar esos nombres.

1. ¿Qué vamos a construir?

Al final tendremos esto:



El correo se enviará:

06:00
09:00
12:00
15:00
18:00
21:00

Y el archivo podría llamarse:

PUNTOS_VENTA_MORA_20260921_060000.csv

Excel puede abrir directamente ese archivo.

2. ¿Por qué vamos a usar CSV y no XLSX?

Porque queremos hacerlo **sin programas externos**.

Oracle puede crear fácilmente:

.csv

y Excel lo abre.

Crear directamente:

.xlsx

desde PL/SQL es considerablemente más complicado.

Por eso primero vamos a hacer:

Oracle

↓

CSV

↓

Correo

↓

Excel

El usuario que recibe el correo solamente hace doble clic en el archivo y Excel lo abre.

3. Antes de comenzar: una aclaración MUY importante

El código de abajo supone estas tablas:

CANAL
DIVISION
TERRITORIO
AGENCIA
PUNTO_VENTA
CLIENTE
SERVICIO
FACTURA
ARREGLO_PAGO

Y aproximadamente estos campos:

CANAL

ID_CANAL

NOMBRE

ESTADO

DIVISION

ID_DIVISION

ID_CANAL

NOMBRE

ESTADO

TERRITORIO

ID_TERRITORIO

ID_DIVISION

NOMBRE

ESTADO

AGENCIA

ID_AGENCIA

ID_TERRITORIO

NOMBRE

ESTADO

PUNTO_VENTA

ID_PUNTO_VENTA

ID_AGENCIA

NOMBRE

ESTADO

MOTIVO_ESTADO

ORIGEN_ESTADO

CLIENTE

ID_CLIENTE

ID_PUNTO_VENTA

NOMBRE

ESTADO

SERVICIO

ID_SERVICIO

ID_CLIENTE

PUNTO_VENTA

...

FACTURA

ID_FACTURA

ID_SERVICIO

ID_CLIENTE

FECHA_VENCIMIENTO

MONTO

SALDO
ESTADO

ARREGLO_PAGO

ID_ARREGLO
ID_CLIENTE
ID_FACTURA
FECHA_INICIO
FECHA_FIN
ESTADO

Si tus tablas tienen otros nombres, no pasa nada.

La estructura del programa seguirá siendo la misma; simplemente cambiaremos los nombres.

4. PRIMER PASO — Crear una carpeta en el servidor Oracle

En el **servidor donde está instalado Oracle Database**, crea:

C:\ORACLE\REPORTES_MORA

No importa si la carpeta está vacía.

Debe quedar:

C:
└─ ORACLE
 └─ REPORTES_MORA

Importante

Si Oracle está instalado en otro servidor, la carpeta debe estar en ese servidor.

No necesariamente en tu computadora.

5. SEGUNDO PASO — Crear el DIRECTORY de Oracle

En PL/SQL Developer abre un SQL Window.

Primero ejecuta:

```
CREATE OR REPLACE DIRECTORY DIR_REPORTES_MORA  
AS 'C:\ORACLE\REPORTES_MORA';
```

Después:

```
GRANT READ, WRITE  
ON DIRECTORY DIR_REPORTES_MORA  
TO TU_USUARIO;
```

Por ejemplo, si tu usuario Oracle es:

VENTAS

sería:

```
GRANT READ, WRITE  
ON DIRECTORY DIR_REPORTES_MORA  
TO VENTAS;
```

6. TERCER PASO — Probar que Oracle puede crear archivos

Antes de hacer todo el sistema, vamos a hacer una prueba pequeña.

Ejecuta:

```
DECLARE  
    v_archivo UTL_FILE.FILE_TYPE;  
BEGIN  
  
    v_archivo := UTL_FILE.FOPEN(  
        'DIR_REPORTES_MORA',  
        'prueba.txt',  
        'W',  
        32767  
    );  
  
    UTL_FILE.PUT_LINE(  
        v_archivo,  
        'Hola, este archivo fue creado por Oracle.'  
    );  
  
    UTL_FILE.FCLOSE(v_archivo);  
  
END;  
/
```

Ahora revisa:

C:\ORACLE\REPORTES_MORA

Debe existir:

prueba.txt

y dentro:

Hola, este archivo fue creado por Oracle.

Si esto funciona

Perfecto.

Ya sabemos que Oracle puede escribir archivos.

7. CUARTO PASO — Crear el historial de bloqueos

Necesitamos guardar cada cambio.

Por ejemplo:

```
PV001
ACTIVE → INACTIVE
Motivo: MORA
Fecha: 21/09/2026 06:00
```

Crea esta tabla:

```
CREATE TABLE HISTORIAL_ESTADO_PV
(
    ID_HISTORIAL          NUMBER GENERATED BY DEFAULT AS IDENTITY,
    ID_PUNTO_VENTA        NUMBER NOT NULL,
    ESTADO_ANTERIOR       VARCHAR2(30),
    ESTADO_NUEVO          VARCHAR2(30),
    MOTIVO                 VARCHAR2(100),
    TIPO_ACCION            VARCHAR2(30),
    ID_CLIENTE             NUMBER,
    ID_FACTURA             NUMBER,
    DIAS_MORA              NUMBER,
    MONTO_MORA             NUMBER(18,2),
    USUARIO                VARCHAR2(100),
    FECHA_ACCION           DATE DEFAULT SYSDATE,
    OBSERVACION            VARCHAR2(500),

    CONSTRAINT PK_HISTORIAL_ESTADO_PV
        PRIMARY KEY (ID_HISTORIAL)
);
```

8. QUINTO PASO — Agregar información al PUNTO_VENTA

Si todavía no tienes estos campos:

```
ALTER TABLE PUNTO_VENTA
ADD (
    MOTIVO_ESTADO VARCHAR2(30),
    ORIGEN_ESTADO VARCHAR2(30)
);
```

La idea es que tengamos:

```
ESTADO
ACTIVE
INACTIVE
```

y además:

```
MOTIVO_ESTADO
-----
NORMAL
MORA
MANUAL
```

y:

```
ORIGEN_ESTADO
-----
AUTOMATICO
MANUAL
```

Esto es importante.

Porque si un empleado bloquea manualmente un punto de venta, el proceso automático no debería desbloquearlo simplemente porque ya no existe mora.

9. SEXTO PASO — Crear una vista de clientes en mora

Ahora vamos a crear una vista.

La vista será como una ventana que dice:

"Muéstrame solamente los clientes que tienen deuda vencida."

Ejemplo:

```
CREATE OR REPLACE VIEW VW_CLIENTES_MORA AS
SELECT
```

```

    pv.id_punto_venta,
    pv.nombre AS punto_venta,

    a.id_agencia,
    a.nombre AS agencia,

    t.id_territorio,
    t.nombre AS territorio,

    d.id_division,
    d.nombre AS division,

    c.id_cliente,
    c.nombre AS cliente,

    f.id_factura,
    f.fecha_vencimiento,

    TRUNC(SYSDATE) - TRUNC(f.fecha_vencimiento) AS dias_mora,

    f.saldo AS monto_mora,

    pv.estado AS estado_punto_venta

FROM CLIENTE c

INNER JOIN PUNTO_VENTA pv
    ON pv.id_punto_venta = c.id_punto_venta

INNER JOIN AGENCIA a
    ON a.id_agencia = pv.id_agencia

INNER JOIN TERRITORIO t
    ON t.id_territorio = a.id_territorio

INNER JOIN DIVISION d
    ON d.id_division = t.id_division

INNER JOIN FACTURA f
    ON f.id_cliente = c.id_cliente

WHERE UPPER(d.nombre) = 'CLIENTES DE MI NEGOCIO'

AND f.fecha_vencimiento < TRUNC(SYSDATE)

AND f.saldo > 0

AND NOT EXISTS
(
    SELECT 1
    FROM ARREGLO_PAGO ap
    WHERE ap.id_cliente = c.id_cliente

    AND (
        ap.id_factura = f.id_factura
        OR ap.id_factura IS NULL
    )

    AND ap.estado = 'ACTIVO'

```

```
AND TRUNC(SYSDATE)
    BETWEEN TRUNC(ap.fecha_inicio)
    AND TRUNC(ap.fecha_fin)
);
```

10. ¿Qué hace esta vista?

Imagina que tienes:

```
Canal
  ↓
División
  ↓
Territorio
  ↓
Agencia
  ↓
Punto de Venta
  ↓
Cliente
  ↓
Factura
```

La consulta recorre todo ese camino.

Además dice:

```
División =
CLIENTES DE MI NEGOCIO
```

Después:

Factura vencida

y:

Saldo > 0

Entonces tenemos una mora.

11. Probar la vista

Ejecuta:

```
SELECT *
FROM VW_CLIENTES_MORA;
```

Si aparecen registros, significa que Oracle está encontrando clientes con mora.

Si no aparece ninguno, puede ser correcto.

Simplemente significa:

Actualmente no hay clientes que cumplan las condiciones.

12. MUY IMPORTANTE: revisar primero esta consulta

Antes de continuar, ejecuta:

```
SELECT
    PUNTO_VENTA,
    CLIENTE,
    ID_FACTURA,
    FECHA_VENCIMIENTO,
    DIAS_MORA,
    MONTO_MORA
FROM VW_CLIENTES_MORA
ORDER BY DIAS_MORA DESC;
```

Deberías ver algo parecido a:

PUNTO_VENTA	CLIENTE	FACTURA	FECHA	DIAS	MONTO
-----	-----	-----	-----	-----	-----
PV001	Empresa ABC	1001	01/09/2026	20	500
PV001	Empresa XYZ	1002	05/09/2026	16	300
PV002	Empresa AAA	1003	10/09/2026	11	800

No continúes hasta que esta consulta sea correcta.

13. SÉPTIMO PASO — Crear el procedimiento que bloquea

Ahora crearemos:

PROCESAR_BLOQUEOS

La regla será:

Si un punto de venta tiene al menos un cliente en mora, se bloquea.

Código:

```
CREATE OR REPLACE PROCEDURE SP_BLOQUEAR_PV_MORA
IS
```

```

BEGIN

    FOR r IN
    (
        SELECT
            id_punto_venta,
            MAX(id_cliente) KEEP
                (DENSE_RANK FIRST ORDER BY dias_mora DESC)
                AS id_cliente,

            MAX(id_factura) KEEP
                (DENSE_RANK FIRST ORDER BY dias_mora DESC)
                AS id_factura,

            MAX(dias_mora) AS dias_mora,
            SUM(monto_mora) AS monto_mora

        FROM VW_CLIENTES_MORA

        GROUP BY id_punto_venta
    )
    LOOP

        UPDATE PUNTO_VENTA
        SET
            estado = 'INACTIVE',
            motivo_estado = 'MORA',
            origen_estado = 'AUTOMATICO'
        WHERE id_punto_venta = r.id_punto_venta
        AND NVL(estado, 'ACTIVE') <> 'INACTIVE';

        IF SQL%ROWCOUNT > 0 THEN

            INSERT INTO HISTORIAL_ESTADO_PV
            (
                ID_PUNTO_VENTA,
                ESTADO_ANTERIOR,
                ESTADO_NUEVO,
                MOTIVO,
                TIPO_ACCION,
                ID_CLIENTE,
                ID_FACTURA,
                DIAS_MORA,
                MONTO_MORA,
                USUARIO,
                FECHA_ACCION,
                OBSERVACION
            )
            VALUES
            (
                r.id_punto_venta,
                'ACTIVE',
                'INACTIVE',
                'MORA',
                'BLOQUEO_AUTOMATICO',
                r.id_cliente,
                r.id_factura,
                r.dias_mora,
                r.monto_mora,

```

```

        USER,
        SYSDATE,
        'Bloqueo automático por mora'
    );

    END IF;

END LOOP;

END;
/

```

14. Probar el bloqueo

Antes de ejecutar:

```

SELECT
    id_punto_venta,
    nombre,
    estado
FROM PUNTO_VENTA;

```

Después ejecuta:

```

BEGIN
    SP_BLOQUEAR_PV_MORA;
    COMMIT;
END;
/

```

Y vuelve a consultar:

```

SELECT
    id_punto_venta,
    nombre,
    estado,
    motivo_estado,
    origen_estado
FROM PUNTO_VENTA;

```

Ahora los que tengan mora deberían aparecer:

```

PV001    INACTIVE    MORA    AUTOMATICO

```

15. Revisar el historial

Ejecuta:

```

SELECT *
FROM HISTORIAL_ESTADO_PV

```

```
ORDER BY FECHA_ACCION DESC;
```

Deberías ver:

```
PV001
ACTIVE
INACTIVE
MORA
BLOQUEO_AUTOMATICO
```

16. OCTAVO PASO — Crear el archivo CSV

Ahora viene una de las partes que más te interesa.

Vamos a crear:

```
SP_GENERAR_REPORTE_MORA
```

Código:

```
CREATE OR REPLACE PROCEDURE SP_GENERAR_REPORTE_MORA
(
    p_nombre_archivo OUT VARCHAR2
)
IS
    v_archivo UTL_FILE.FILE_TYPE;
BEGIN
    /*
        Creamos el nombre del archivo.

        Ejemplo:

        PUNTOS_VENTA_MORA_20260921_060000.csv
    */
    p_nombre_archivo :=
        'PUNTOS_VENTA_MORA_' ||
        TO_CHAR(SYSDATE, 'YYYYMMDD_HH24MISS') ||
        '.csv';

    /*
        Abrimos el archivo.
    */
    v_archivo :=
        UTL_FILE.FOPEN(
            'DIR_REPORTES_MORA',
            p_nombre_archivo,
            'W',
            32767
        );
```

```

/*
    Escribimos los encabezados.
*/

UTL_FILE.PUT_LINE(
    v_archivo,
    'Punto de Venta, Agencia, Territorio, Division, Cliente, Factura, Fecha
Vencimiento, Dias Mora, Monto Mora, Estado'
);

/*
    Escribimos los registros.
*/

FOR r IN
(
    SELECT
        punto_venta,
        agencia,
        territorio,
        division,
        cliente,
        id_factura,
        TO_CHAR(
            fecha_vencimiento,
            'DD/MM/YYYY'
        ) AS fecha_vencimiento,
        dias_mora,
        monto_mora,
        estado_punto_venta
    FROM VW_CLIENTES_MORA
    ORDER BY
        punto_venta,
        dias_mora DESC
)
LOOP

    UTL_FILE.PUT_LINE(
        v_archivo,

        ''' ||
        REPLACE(r.punto_venta, '''', '""') ||
        ''', ' ||

        ''' ||
        REPLACE(r.agencia, '''', '""') ||
        ''', ' ||

        ''' ||
        REPLACE(r.territorio, '''', '""') ||
        ''', ' ||

        ''' ||
        REPLACE(r.division, '''', '""') ||
        ''', ' ||

```

```

        ''' ||
        REPLACE(r.cliente, '''', ''') ||
        ''', ' ' ||

        ''' ||
        r.id_factura ||
        ''', ' ' ||

        ''' ||
        r.fecha_vencimiento ||
        ''', ' ' ||

        r.dias_mora ||
        ', ' ||

        TO_CHAR(
            r.monto_mora,
            'FM9999999999999999D00',
            'NLS_NUMERIC_CHARACTERS=','.', ''
        ) ||
        ', ' ||

        ''' ||
        r.estado_punto_venta ||
        ''''
    );

END LOOP;

/*
   Cerramos el archivo.
*/

UTL_FILE.FCLOSE(v_archivo);

EXCEPTION

    WHEN OTHERS THEN

        IF UTL_FILE.IS_OPEN(v_archivo) THEN
            UTL_FILE.FCLOSE(v_archivo);
        END IF;

        RAISE;

END;
/

```

17. ¿Qué acaba de hacer Oracle?

Si ejecutas:

```
DECLARE
```

```

v_archivo VARCHAR2(500);

BEGIN

    SP_GENERAR_REPORTE_MORA(v_archivo);

    DBMS_OUTPUT.PUT_LINE(
        'Archivo creado: ' || v_archivo
    );

END;
/

```

Oracle puede crear:

PUNTOS_VENTA_MORA_20260921_231500.csv

en:

C:\ORACLE\REPORTES_MORA

18. Abrir el CSV con Excel

Haz doble clic en:

PUNTOS_VENTA_MORA_20260921_231500.csv

Deberías tener:

Punto de Venta	Agencia	Territorio	División	Cliente	Factura	Fecha	Días	Monto	Estado
PV001	Agencia 1	Managua	Cientes..	Empresa ABC	1001	01/09/20 26	20	500	INACTIVE

19. NOVENO PASO — Configurar el correo

Aquí necesitamos saber algo que solamente tu empresa puede proporcionar:

Servidor SMTP
Puerto SMTP
Usuario
Contraseña

Por ejemplo, **NO uses estos datos literalmente:**

SMTP: smtp.miempresa.com
Puerto: 25
Usuario: reportes@miempresa.com

Son solamente ejemplos.

20. Una advertencia importante sobre SMTP

La forma más sencilla de hacerlo es utilizar un **servidor SMTP interno que permita a Oracle enviar correos**.

Por ejemplo:

```
Oracle
  ↓
smtp.empresa.local
  ↓
Internet/correo corporativo
```

Si tu servidor exige:

587 + STARTTLS

o:

465 + SSL

la configuración de Oracle es más complicada porque probablemente necesitaremos configurar un **Oracle Wallet**.

Por eso, antes de intentar el correo, primero debemos confirmar cómo funciona el SMTP de tu empresa.

21. DÉCIMO PASO — Crear el procedimiento de correo

Supongamos, para el ejemplo, que tenemos:

```
SMTP_SERVER = smtp.miempresa.com
SMTP_PORT   = 25
REMITENTE   = reportes@miempresa.com
DESTINATARIO = jefe@miempresa.com
```

El siguiente procedimiento utiliza UTL_SMTP.

```
CREATE OR REPLACE PROCEDURE SP_ENVIAR_REPORTE_MORA
(
  p_nombre_archivo IN VARCHAR2
)
IS
  v_smtp_server VARCHAR2(200) := 'smtp.miempresa.com';
  v_smtp_port   NUMBER := 25;
```

```

v_remitente VARCHAR2(200) :=
    'reportes@miempresa.com';

v_destinatario VARCHAR2(200) :=
    'jefe@miempresa.com';

v_mail UTL_SMTP.CONNECTION;

v_boundary VARCHAR2(100) :=
    '----=_MORA_BOUNDARY_123456';

v_linea VARCHAR2(32767);

v_archivo UTL_FILE.FILE_TYPE;

```

BEGIN

```

/*
  Abrimos conexión con el servidor SMTP.
*/

```

```

v_mail :=
    UTL_SMTP.OPEN_CONNECTION(
        v_smtp_server,
        v_smtp_port
    );

```

```

/*
  Nos identificamos.
*/

```

```

UTL_SMTP.HELO(
    v_mail,
    v_smtp_server
);

```

```

/*
  Remitente.
*/

```

```

UTL_SMTP.MAIL(
    v_mail,
    v_remitente
);

```

```

/*
  Destinatario.
*/

```

```

UTL_SMTP.RCPT(
    v_mail,
    v_destinatario
);

```

```

/*
    Comenzamos el correo.
*/

UTL_SMTP.OPEN_DATA(v_mail);

/*
    Encabezados.
*/

UTL_SMTP.WRITE_DATA(
    v_mail,
    'From: ' || v_remitente || UTL_TCP.CRLF
);

UTL_SMTP.WRITE_DATA(
    v_mail,
    'To: ' || v_destinatario || UTL_TCP.CRLF
);

UTL_SMTP.WRITE_DATA(
    v_mail,
    'Subject: Reporte de Puntos de Venta en Mora' ||
    UTL_TCP.CRLF
);

UTL_SMTP.WRITE_DATA(
    v_mail,
    'MIME-Version: 1.0' ||
    UTL_TCP.CRLF
);

UTL_SMTP.WRITE_DATA(
    v_mail,
    'Content-Type: multipart/mixed; boundary="' ||
    v_boundary ||
    '"' ||
    UTL_TCP.CRLF
);

UTL_SMTP.WRITE_DATA(
    v_mail,
    UTL_TCP.CRLF
);

/*
    Parte de texto del correo.
*/

UTL_SMTP.WRITE_DATA(
    v_mail,
    '--' ||
    v_boundary ||
    UTL_TCP.CRLF
);

UTL_SMTP.WRITE_DATA(

```

```

        v_mail,
        'Content-Type: text/plain; charset="UTF-8"' ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'Buenos días.' ||
        UTL_TCP.CRLF ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'Se adjunta el reporte actualizado de los puntos de venta que presentan
clientes en mora.' ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        UTL_TCP.CRLF
    );

    /*
    Parte correspondiente al archivo.
    */

    UTL_SMTP.WRITE_DATA(
        v_mail,
        '--' ||
        v_boundary ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'Content-Type: text/csv; name="' ||
        p_nombre_archivo ||
        '"' ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'Content-Disposition: attachment; filename="' ||
        p_nombre_archivo ||
        '"' ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,

```

```

        'Content-Transfer-Encoding: base64' ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        UTL_TCP.CRLF
    );

    /*
    Abrimos el archivo generado.
    */

    v_archivo :=
        UTL_FILE.FOPEN(
            'DIR_REPORTES_MORA',
            p_nombre_archivo,
            'R',
            32767
        );

    /*
    Leemos el archivo y lo enviamos.
    */

LOOP

    BEGIN

        UTL_FILE.GET_LINE(
            v_archivo,
            v_linea
        );

        /*
        Para una primera versión sencilla
        enviamos cada línea codificada.
        */

        UTL_SMTP.WRITE_DATA(
            v_mail,
            UTL_RAW.CAST_TO_VARCHAR2(
                UTL_ENCODE.BASE64_ENCODE(
                    UTL_RAW.CAST_TO_RAW(
                        v_linea
                    )
                )
            ) ||
            UTL_TCP.CRLF
        );

    EXCEPTION

        WHEN NO_DATA_FOUND THEN
            EXIT;

    END;

```

```

END LOOP;

/*
  Cerramos archivo.
*/

UTL_FILE.FCLOSE(v_archivo);

/*
  Finalizamos el correo.
*/

UTL_SMTP.WRITE_DATA(
  v_mail,
  UTL_TCP.CRLF ||
  '--' ||
  v_boundary ||
  '--' ||
  UTL_TCP.CRLF
);

UTL_SMTP.CLOSE_DATA(v_mail);

UTL_SMTP.QUIT(v_mail);

EXCEPTION

  WHEN OTHERS THEN

    IF UTL_FILE.IS_OPEN(v_archivo) THEN
      UTL_FILE.FCLOSE(v_archivo);
    END IF;

    BEGIN
      UTL_SMTP.QUIT(v_mail);
    EXCEPTION
      WHEN OTHERS THEN
        NULL;
    END;

    RAISE;

END;
/

```

22. Pero aquí hay una mejora importante

El procedimiento anterior sirve para explicar la estructura, pero **no quiero que lo pongas en producción todavía**.

¿Por qué?

Porque el envío de un archivo mediante MIME/base64 debe manejar correctamente los bloques y saltos de línea, especialmente si el CSV puede ser grande.

Para un reporte de pocos registros puede funcionar, pero para un sistema empresarial prefiero construir una versión más robusta después de saber:

- versión de Oracle;
 - tamaño máximo aproximado del archivo;
 - servidor SMTP;
 - si utiliza autenticación;
 - si utiliza TLS.
-

23. Primero probamos el SMTP SIN archivo

Antes de enviar el Excel/CSV, hacemos una prueba todavía más pequeña:

"Oracle, mándame un correo que diga Hola."

Esto es muy importante.

No debemos intentar resolver:

```
SMTP
+
archivo
+
base64
+
Scheduler
+
mora
```

todo al mismo tiempo.

Primero:

Oracle → correo

24. Procedimiento de prueba de correo

```
CREATE OR REPLACE PROCEDURE SP_PRUEBA_CORREO
IS
```

```
    v_mail UTL_SMTP.CONNECTION;

    v_smtp_server VARCHAR2(200) :=
        'smtp.miempresa.com';
```

```

v_smtp_port NUMBER := 25;

v_remitente VARCHAR2(200) :=
    'reportes@miempresa.com';

v_destinatario VARCHAR2(200) :=
    'jefe@miempresa.com';

BEGIN

    v_mail :=
        UTL_SMTP.OPEN_CONNECTION(
            v_smtp_server,
            v_smtp_port
        );

    UTL_SMTP.HELO(
        v_mail,
        v_smtp_server
    );

    UTL_SMTP.MAIL(
        v_mail,
        v_remitente
    );

    UTL_SMTP.RCPT(
        v_mail,
        v_destinatario
    );

    UTL_SMTP.OPEN_DATA(v_mail);

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'From: ' || v_remitente ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'To: ' || v_destinatario ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'Subject: PRUEBA ORACLE' ||
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        UTL_TCP.CRLF
    );

    UTL_SMTP.WRITE_DATA(
        v_mail,
        'Este es un correo de prueba enviado desde Oracle.' ||

```

```
        UTL_TCP.CRLF
    );

    UTL_SMTP.CLOSE_DATA(v_mail);

    UTL_SMTP.QUIT(v_mail);

END;
/
```

Luego:

```
BEGIN
    SP_PRUEBA_CORREO;
END;
/
```

Si recibes el correo:

Perfecto.

Ya sabemos que Oracle puede enviar correos.

25. Si aparece un error de ACL

Puedes encontrar algo parecido a:

```
ORA-24247: network access denied by access control list
```

Eso significa:

Oracle todavía no tiene permiso para conectarse al servidor SMTP.

Eso no significa que el código esté mal.

El DBA debe configurar el permiso de red.

Por ejemplo, en versiones compatibles con ACL se configura mediante
DBMS_NETWORK_ACL_ADMIN.

La configuración exacta depende de tu versión de Oracle, por eso **no quiero que ejecutes comandos de ACL a ciegas.**

Primero averigua:

```
SELECT *
FROM V$VERSION;
```

Guarda el resultado.

26. UNDÉCIMO PASO — Crear el proceso completo

Cuando ya tengamos:

- ✓ Detectar mora
- ✓ Bloquear
- ✓ Historial
- ✓ Generar CSV
- ✓ Enviar correo

creamos:

```
CREATE OR REPLACE PROCEDURE SP_PROCESAR_MORA
IS
```

```
    v_nombre_archivo VARCHAR2(500);
```

```
BEGIN
```

```
    /*
        PASO 1
        Bloquear puntos de venta con mora.
    */
```

```
    SP_BLOQUEAR_PV_MORA;
```

```
    /*
        PASO 2
        Crear el reporte.
    */
```

```
    SP_GENERAR_REPORTE_MORA(
        v_nombre_archivo
    );
```

```
    /*
        PASO 3
        Enviar el reporte.
    */
```

```
    SP_ENVIAR_REPORTE_MORA(
        v_nombre_archivo
    );
```

```
    /*
        Guardamos los cambios.
    */
```

```
    COMMIT;
```

```
END;
/
```

Ahora tenemos una sola orden:

```
BEGIN
    SP_PROCESAR_MORA;
END;
/
```

Y Oracle hace todo.

27. DUODÉCIMO PASO — Crear el Scheduler

Ahora vamos a decirle a Oracle:

"Ejecuta este procedimiento cada 3 horas."

Primero comprobamos si ya existe el Job:

```
SELECT
    job_name,
    enabled,
    state
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_CONTROL_MORA';
```

Si no existe, creamos:

```
BEGIN
    DBMS_SCHEDULER.CREATE_JOB
    (
        job_name =>
            'JOB_CONTROL_MORA',

        job_type =>
            'STORED_PROCEDURE',

        job_action =>
            'SP_PROCESAR_MORA',

        start_date =>
            SYSTIMESTAMP,

        repeat_interval =>
            'FREQ=DAILY;BYHOUR=6,9,12,15,18,21;BYMINUTE=0;BYSECOND=0',

        enabled =>
            FALSE,

        auto_drop =>
            FALSE,

        comments =>
            'Control automático de mora y reporte de puntos de venta'
    );
```

```
END;  
/
```

Observa que puse:

`enabled => FALSE`

Esto es **intencional**.

Todavía no queremos que empiece a ejecutarse automáticamente.

Primero vamos a probar.

28. Probar manualmente el Job

Podemos decirle:

```
BEGIN  
  
    DBMS_SCHEDULER.RUN_JOB(  
        'JOB_CONTROL_MORA',  
        TRUE  
    );  
  
END;  
/
```

Esto significa:

"Oracle, ejecuta ahora el proceso."

No necesitamos esperar hasta las 06:00.

29. Revisar si el Job funcionó

Ejecuta:

```
SELECT  
    job_name,  
    status,  
    actual_start_date,  
    run_duration,  
    error#  
FROM USER_SCHEDULER_JOB_RUN_DETAILS  
WHERE job_name = 'JOB_CONTROL_MORA'  
ORDER BY actual_start_date DESC;
```

Si todo salió bien, deberías ver:

JOB_CONTROL_MORA
SUCCEEDED

30. Si todo funciona, activamos el Job

Ahora sí:

```
BEGIN
    DBMS_SCHEDULER.ENABLE(
        'JOB_CONTROL_MORA'
    );
END;
/
```

Y revisamos:

```
SELECT
    job_name,
    enabled,
    state,
    next_run_date
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_CONTROL_MORA';
```

Debería aparecer:

```
JOB_CONTROL_MORA
TRUE
SCHEDULED
```

31. ¿Qué pasará ahora?

Oracle automáticamente hará:

```
06:00
↓
SP_PROCESAR_MORA
↓
Buscar mora
↓
Bloquear PV
↓
Registrar historial
↓
Crear CSV
↓
Enviar correo
```

Después:

09:00

↓

SP_PROCESAR_MORA

Después:

12:00

↓

SP_PROCESAR_MORA

etc.

32. ¿Qué pasa si NO hay mora?

Esto es importante.

Supongamos que a las 12:00:

Clientes con mora = 0

El sistema puede generar:

PUNTOS_VENTA_MORA_20260921_120000.csv

con solamente los encabezados.

Yo prefiero que **sí se envíe el correo**, diciendo:

No existen puntos de venta con clientes en mora

Pero podemos modificarlo para que:

0 morosos → no enviar correo

si ese es el comportamiento que prefieres.

33. ¿Qué pasa si hay 3 clientes morosos en el mismo punto de venta?

Supongamos:

PV001

- └─ Cliente A → 10 días
- └─ Cliente B → 20 días
- └─ Cliente C → 5 días

El punto de venta se bloquea **una sola vez**:

PV001 → INACTIVE

Pero el reporte puede mostrar los tres clientes:

PV001 | Cliente A | 10 días

PV001 | Cliente B | 20 días

PV001 | Cliente C | 5 días

Eso es precisamente lo que queremos.

34. ¿Qué pasa si ya está bloqueado?

Supongamos:

PV001 = INACTIVE

y sigue teniendo mora.

El proceso NO debe hacer:

INACTIVE → INACTIVE

ni crear un nuevo historial cada 3 horas.

El código:

```
AND NVL(estado, 'ACTIVE') <> 'INACTIVE'
```

evita eso.

Por tanto:

06:00

ACTIVE → INACTIVE

se registra.

Pero:

09:00

INACTIVE → INACTIVE

no genera otro bloqueo.

35. ¿Qué pasa cuando paga?

Supongamos:

PV001
Cliente ABC
Factura 1001
Saldo = 500

Después paga todo:

Saldo = 0

Entonces deja de aparecer en:

VW_CLIENTES_MORA

Pero todavía necesitamos el procedimiento de **desbloqueo automático**.

Ese será el siguiente paso del proyecto.

La regla será:

Si:

PUNTO_VENTA.estado = INACTIVE
Y
motivo_estado = MORA
Y
ya no tiene clientes morosos

Entonces:

ACTIVE

Pero si:

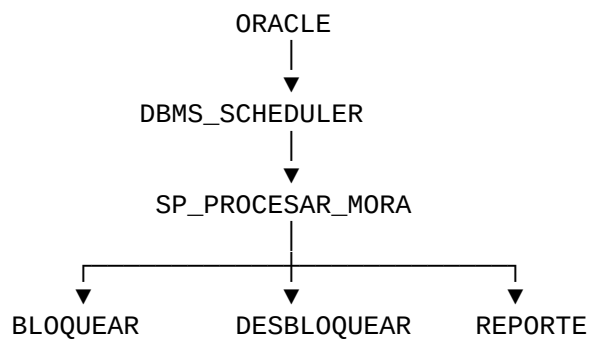
origen_estado = MANUAL

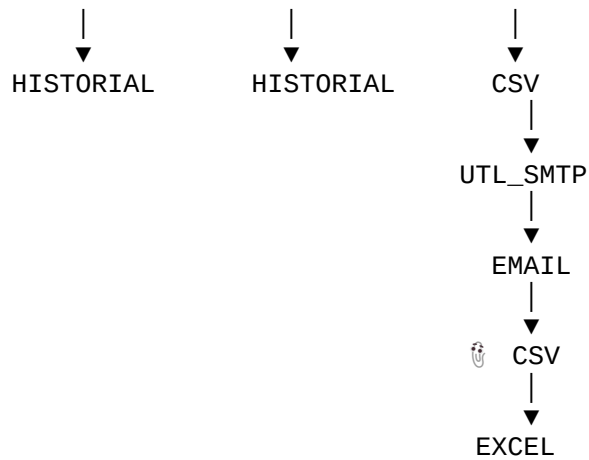
no se toca.

Esto evita que Oracle desbloquee accidentalmente un punto de venta bloqueado manualmente.

36. La versión completa que tendremos

Finalmente la arquitectura será:





37. ¿En qué orden debes hacerlo?

No copies todo y ejecutes todo de una sola vez.

Hazlo exactamente en este orden:

Paso 1

Crear carpeta:

C:\ORACLE\REPORTES_MORA

Paso 2

Crear:

DIR_REPORTES_MORA

Paso 3

Probar:

UTL_FILE

y comprobar que aparece:

prueba.txt

Paso 4

Crear:

HISTORIAL_ESTADO_PV

Paso 5

Agregar:

```
MOTIVO_ESTADO  
ORIGEN_ESTADO
```

Paso 6

Crear:

```
VW_CLIENTES_MORA
```

Paso 7

Ejecutar:

```
SELECT *  
FROM VW_CLIENTES_MORA;
```

Y comprobar que los datos son correctos.

Paso 8

Crear:

```
SP_BLOQUEAR_PV_MORA
```

Paso 9

Probar bloqueo manualmente.

Paso 10

Comprobar:

```
HISTORIAL_ESTADO_PV
```

Paso 11

Crear:

```
SP_GENERAR_REPORTE_MORA
```

Paso 12

Generar el CSV manualmente.

Paso 13

Abrirlo con Excel.

Paso 14

Configurar SMTP.

Paso 15

Probar primero un correo **sin archivo**.

Paso 16

Probar correo **con CSV adjunto**.

Paso 17

Crear:

SP_PROCESAR_MORA

Paso 18

Crear Scheduler:

JOB_CONTROL_MORA

Paso 19

Ejecutarlo manualmente.

Paso 20

Revisar:

USER_SCHEDULER_JOB_RUN_DETAILS

Paso 21

Finalmente activar:

DBMS_SCHEDULER.ENABLE

38. Algo que quiero que hagamos antes de que ejecutes el código

Como me dijiste que eres nuevo y quieres **hacerlo bien sin equivocarte**, yo **no te recomiendo que ejecutes todavía la VW_CLIENTES_MORA ni los procedimientos de bloqueo**.

La razón es sencilla:

Los nombres que he utilizado:

CLIENTE
PUNTO_VENTA
FACTURA
ARREGLO_PAGO

son los nombres de referencia que venimos utilizando, pero **tu base real puede tener nombres y relaciones diferentes**.

Y en tu caso hay un detalle especialmente importante:

Me indicaste anteriormente que **SERVICIOS** contiene el campo del punto de venta y que solamente deben considerarse los puntos de venta de la división "**CLIENTES DE MI NEGOCIO**".

Por eso, para no inventar la relación, lo correcto es construir la consulta usando **tus tablas reales**.

Haz esto ahora en PL/SQL Developer

Ejecuta:

```
DESC CANAL;  
DESC DIVISION;  
DESC TERRITORIO;  
DESC AGENCIA;  
DESC PUNTO_VENTA;  
DESC CLIENTE;  
DESC SERVICIOS;  
DESC FACTURA;  
DESC ARREGLO_PAGO;
```

Si alguna tabla tiene otro nombre, ejecuta el DESC con el nombre real.

También necesito saber la versión de Oracle:

```
SELECT * FROM V$VERSION;
```

Con esos resultados podemos hacer el siguiente paso usando exactamente tus tablas reales, y te puedo entregar el PACKAGE **PKG_CONTROL_MORA** completo —bloqueo, desbloqueo, historial, generación del CSV, envío del correo y Scheduler— sin que tengas que adivinar qué nombre de columna debes cambiar.